

GRADUAÇÃO

 FIAP

MOBILE APPLICATION DEVELOPMENT

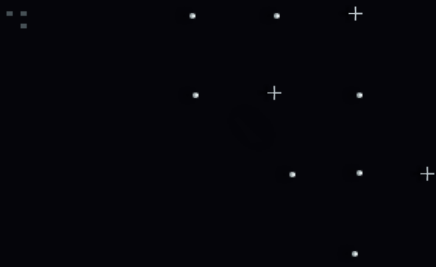
ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

FIAP

AULA 08

**Nem todo app precisa de uma
API própria**

Cloud Firestore como backend gerenciado



“O que torna um recurso verdadeiramente estratégico não é sua ubiquidade, mas sua escassez.”

— *Nicholas G. Carr, IT Doesn't Matter, 2003, tradução livre*



O QUE MERECE SER CONSTRUÍDO POR NÓS?

Quais partes do backend expressam o diferencial do aplicativo e quais podem ser fornecidas por uma plataforma?

CAPACIDADES DE UM BACKEND

Específicas da aplicação	Recorrentes em muitos sistemas
Regras do domínio	Identificação de usuários
Fluxos próprios do produto	Persistência de dados
Cálculos exclusivos	Armazenamento de arquivos
Decisões do negócio	Controle de acesso
Integrações particulares	Envio de notificações

AS DUAS CATEGORIAS SÃO NECESSÁRIAS, MAS POSSUEM VALORES ESTRATÉGICOS DIFERENTES.

CAPACIDADES RECORRENTES

Uma capacidade recorrente:

- aparece em muitos sistemas
- resolve um problema conhecido
- possui soluções amplamente disponíveis
- pode ser oferecida como serviço

Esse tipo de capacidade pode se tornar uma **commodity tecnológica**.

COMMODITY SIGNIFICA COMUM E DISPONÍVEL, NÃO SIMPLES OU IRRELEVANTE.

O CUSTO DE UMA IMPLEMENTAÇÃO PRÓPRIA

Entregar a primeira versão representa apenas uma parte do trabalho.

Momento	Responsabilidade
Construção	Implementar e testar
Entrada em produção	Implantar e configurar
Operação	Monitorar disponibilidade e desempenho
Evolução	Corrigir falhas e atualizar dependências
+Crescimento	Ajustar capacidade e custos

A EQUIPE ASSUME ESSE COMPROMISSO ENQUANTO A CAPACIDADE PERMANECER EM USO.

CAPACIDADES CONSUMIDAS COMO SERVIÇO

Já delegamos partes importantes de muitos sistemas:

- provedores enviam e-mails
- gateways processam pagamentos
- plataformas fornecem mapas
- serviços armazenam arquivos

A aplicação integra uma capacidade que já está em operação.

O MESMO RACIOCÍNIO PODE SER APLICADO A PARTES DO BACKEND.

BACKEND AS A SERVICE

Backend as a Service, ou BaaS, é uma categoria de plataformas que fornece capacidades de backend por meio de SDKs e APIs.

Uma plataforma pode oferecer:

Serviço	Capacidade
Identidade	Criar contas e autenticar usuários
Banco de dados	Armazenar e consultar informações
Arquivos	Guardar imagens e documentos
Funções	Executar código remotamente
Eventos	Comunicar alterações
Notificações	Enviar mensagens aos dispositivos

SDK E SERVIÇO REMOTO

Uma biblioteca comum executa seu código dentro da aplicação.

Um BaaS possui serviços em execução na infraestrutura do provedor.

Aplicativo

SDK instalado

chamadas para o serviço

Plataforma

servidores

armazenamento

execução das operações

O SDK CONECTA A APLICAÇÃO AO BACKEND REMOTO.

O BACKEND CONTINUA EXISTINDO

API própria	Backend as a Service
Aplicativo	Aplicativo
API mantida pela equipe	SDK ou API da plataforma
Banco configurado pela equipe	Serviço gerenciado
Infraestrutura operada pela equipe	Infraestrutura operada pelo provedor

A mudança está na responsabilidade por construir e operar cada parte.

DIVISÃO DE RESPONSABILIDADES

Plataforma	Equipe da aplicação
Executar os serviços	Definir o modelo de dados
Manter a infraestrutura disponível	Implementar as regras do domínio
Atualizar a plataforma	Configurar o controle de acesso
Oferecer mecanismos de escalabilidade	Acompanhar consumo e custos
Persistir os dados	Validar os dados utilizados

UTILIZAR UM SERVIÇO GERENCIADO REDUZ TRABALHO OPERACIONAL, MAS NÃO ELIMINA DECISÕES DE ARQUITETURA.

LIMITES DA DELEGAÇÃO

A adoção de um BaaS também cria compromissos:

- dependência dos recursos oferecidos pela plataforma
- consultas condicionadas ao modelo e aos índices disponíveis
- cobrança baseada no uso dos serviços
- migração mais trabalhosa para outra tecnologia
- necessidade de configurar corretamente as regras de acesso

Esses fatores fazem parte da decisão entre construir e contratar.

BAAS E API PRÓPRIA PODEM COEXISTIR

Uma aplicação pode utilizar cada alternativa onde ela oferece mais valor.

Aplicação

BaaS

autenticação

armazenamento

dados com acesso direto

API própria

regras exclusivas

integrações privadas

operações privilegiadas

A ESCOLHA PODE OCORRER CAPACIDADE POR CAPACIDADE.

FIREBASE E CLOUD FIRESTORE

Firebase é uma plataforma que reúne diferentes serviços de backend.

Cloud Firestore é o serviço de banco de dados orientado a documentos dessa plataforma.

Ele oferece:

- armazenamento de documentos
- consultas
- controle de acesso
- sincronização de alterações
- integração com SDKs para aplicações móveis e web

NESTA AULA, O FOCO SERÁ A LEITURA DE DOCUMENTOS POR MEIO DE CONSULTAS.

ACESSO DIRETO E CONTROLE DE ACESSO

O aplicativo pode se comunicar diretamente com o Firestore, mas cada elemento possui uma responsabilidade diferente.

Elemento	Responsabilidade
Configuração do Firebase	Identificar o projeto e a aplicação
Firebase Authentication	Identificar o usuário atual
Security Rules	Autorizar ou rejeitar a operação
Cloud Firestore	Executar a operação permitida

A configuração presente no aplicativo não concede acesso irrestrito aos dados.

AS REGRAS DE AUTORIZAÇÃO SERÃO APROFUNDADAS NA PRÓXIMA AULA.

COMO O FIRESTORE ORGANIZA OS DADOS?

Antes de consultar informações, precisamos entender como elas são armazenadas.

MODELO ORIENTADO A DOCUMENTOS

Cloud Firestore organiza os dados utilizando:

- **coleções**, que agrupam documentos
- **documentos**, que representam unidades de informação
- **campos**, que armazenam valores
- **identificadores**, que distinguem os documentos

Exemplo:

```
posts
├── 7qA2fLm9
│   ├── content: "Estudando bancos NoSQL"
│   ├── authorId: "user-42"
│   └── createdAt: Timestamp
```

IDENTIDADE E CAMINHO

O caminho identifica a localização de um documento.

```
posts/7qA2fLm9
```

Parte	Significado
posts	Coleção
7qA2fLm9	Identificador do documento
posts/7qA2fLm9	Caminho completo

Dois documentos da mesma coleção não podem possuir o mesmo identificador.

DOCUMENTO E LINHA RELACIONAL

Banco relacional	Banco orientado a documentos
Tabelas organizam linhas	Coleções organizam documentos
Colunas definem a estrutura	Campos representam os valores
Chave primária identifica a linha	Caminho identifica o documento
Relacionamentos permitem junções	Documentos podem conter mapas e listas
O banco aplica o esquema da tabela	A aplicação controla a consistência dos campos

UM DOCUMENTO PODE REPRESENTAR UMA ESTRUTURA MAIS PRÓXIMA DO OBJETO UTILIZADO PELA APLICAÇÃO.

FLEXIBILIDADE E CONTRATO

O Firestore permite que documentos da mesma coleção tenham campos diferentes.

Essa flexibilidade também permite documentos incompatíveis:

```
posts/post-1
  content: string
  authorId: string
  createdAt: Timestamp
```

```
posts/post-2
  content: number
  createdAt: string
```

A AUSÊNCIA DE UM ESQUEMA RÍGIDO AUMENTA A RESPONSABILIDADE DA APLICAÇÃO PELA VALIDAÇÃO.

O MODELO CONSIDERA AS CONSULTAS

Um modelo de documentos precisa representar o domínio e permitir as consultas exigidas pelas telas.

Necessidade da interface	Consequência no modelo
Mostrar as publicações recentes	Todos os documentos precisam de <code>createdAt</code>
Mostrar publicações de um autor	Os documentos precisam de <code>authorId</code>
Apresentar o conteúdo	Os documentos precisam de <code>content</code>
Ordenar pelo horário	O campo precisa ser consultável e indexado

**A ESTRUTURA DOS DOCUMENTOS INFLUENCIA QUAIS CONSULTAS
PODEM SER EXECUTADAS.**

COMO ESCOLHEMOS QUAIS DOCUMENTOS RECUPERAR?

Uma consulta descreve o conjunto de documentos necessário para responder a uma necessidade da aplicação.

DA NECESSIDADE À CONSULTA

Considere a seguinte necessidade:

Apresentar as dez publicações mais recentes.

Decisão	Definição
Origem	Coleção <code>posts</code>
Ordem	<code>createdAt</code> decrescente
Quantidade	Dez documentos

AGORA PRECISAMOS EXPRESSAR ESSAS DECISÕES UTILIZANDO O SDK.

A INSTÂNCIA DO FIRESTORE

Antes de referenciar uma coleção, o código precisa obter acesso ao serviço Firestore associado à aplicação Firebase.

```
import { getFirestore } from "firebase/firestore";  
  
export const db = getFirestore(app);
```

`app` representa a aplicação Firebase configurada.

`db` representa a instância do Cloud Firestore associada ao mesmo projeto.

`getFirestore(app)` não cria outro projeto nem reinicializa o Firebase.

REFERÊNCIA DA COLEÇÃO

O primeiro passo consiste em indicar onde estão os documentos.

```
import { collection } from "firebase/firestore";  
  
const postsCollection = collection(db, "posts");
```

`postsCollection` é uma referência.

Ela representa a localização da coleção `posts`.

NENHUMA LEITURA FOI EXECUTADA ATÉ ESSE MOMENTO.

DESCRIÇÃO DA CONSULTA

A consulta especifica quais documentos são necessários.

```
import { limit, orderBy, query } from "firebase/firestore";

const recentPostsQuery = query(
  postsCollection,
  orderBy("createdAt", "desc"),
  limit(10),
);
```

Essa consulta descreve:

- a coleção utilizada
- a ordem pelo campo `createdAt`
- o limite de dez documentos

DESCREVER UMA CONSULTA NÃO PRODUZ RESULTADOS.

EXECUÇÃO DA LEITURA

`getDocs` executa a consulta.

```
import { getDocs } from "firebase/firestore";

const snapshot = await getDocs(recentPostsQuery);
```

O retorno é um `QuerySnapshot`.

Ele representa o conjunto de documentos encontrado durante aquela execução.

REFERÊNCIA, CONSULTA E RESULTADO

Elemento	Pergunta respondida
<code>collection()</code>	Onde estão os documentos?
<code>query()</code>	Quais documentos queremos?
<code>orderBy()</code>	Em qual ordem?
<code>limit()</code>	Quantos documentos?
<code>getDocs()</code>	Quando executar a leitura?
<code>QuerySnapshot</code>	Qual foi o resultado?

O CONTEÚDO DO QUERYSNAPSHOT

Um `QuerySnapshot` oferece informações sobre o resultado:

```
snapshot.empty;  
snapshot.size;  
snapshot.docs;
```

Propriedade	Conteúdo
<code>empty</code>	Indica se nenhum documento foi encontrado
<code>size</code>	Informa a quantidade de documentos
<code>docs</code>	Contém os documentos retornados

Cada item de `docs` é um `QueryDocumentSnapshot`.

UMA FOTOGRAFIA DOS DADOS

`getDocs` encerra sua operação depois de produzir o resultado.

Momento	Estado
10:00	<code>getDocs</code> retorna dez documentos
10:01	Outro cliente cria um documento
10:02	O snapshot anterior continua com os mesmos dez documentos
10:03	Uma nova execução produz outro snapshot

O SNAPSHOT REPRESENTA O RESULTADO DISPONÍVEL EM UMA EXECUÇÃO ESPECÍFICA.

UMA NOVA EXECUÇÃO

O aplicativo pode solicitar outra leitura quando precisar de dados atualizados.

Com TanStack Query, isso pode acontecer por:

- chamada explícita a `refetch`
- nova montagem de uma consulta considerada desatualizada
- reconexão ou retorno do foco, conforme a configuração

Cada execução de `getDocs` continua produzindo um novo snapshot.

A assinatura contínua representa outra estratégia e será estudada posteriormente.

IDENTIFICADOR E DADOS

O identificador e os campos vêm de partes diferentes do documento.

```
const posts = snapshot.docs.map((document) => ({
  ...document.data(),
  id: document.id,
}));
```

Expressão	Resultado
document.id	Identificador do documento
document.data()	Campos armazenados
Objeto resultante	Modelo utilizado pela aplicação

TYPESCRIPT E DADOS EXTERNOS

Uma conversão de tipo informa ao compilador como tratar um valor:

```
const data = document.data() as Post;
```

Essa instrução não verifica o conteúdo recebido.

Um documento pode continuar contendo:

- campos ausentes
- valores de tipos diferentes
- datas armazenadas como texto
- estruturas inesperadas

DADOS EXTERNOS PRECISAM DE VALIDAÇÃO DURANTE A EXECUÇÃO.

VALIDAÇÃO COM ZOD

O schema define o contrato esperado pela aplicação.

```
import { Timestamp } from "firebase/firestore";
import { z } from "zod";

const postSchema = z.object({
  id: z.string(),
  content: z.string(),
  authorId: z.string(),
  createdAt: z.instanceof(Timestamp),
});
```

Cada documento pode ser validado durante a conversão:

```
const posts = snapshot.docs.map((document) =>
  postSchema.parse({
    ...document.data(),
    id: document.id,
  }),
);
```

Um documento inválido transforma a leitura em uma falha tratável.

FIRESTORE E TANSTACK QUERY

As duas tecnologias utilizam o termo “query”, mas possuem responsabilidades diferentes.

Tecnologia	Responsabilidade
Firestore	Descrever e executar a seleção remota
TanStack Query	Controlar execução, cache e estados
Zod	Validar os documentos recebidos
React Native	Apresentar o resultado

Exemplo de integração:

```
export function useRecentPostsQuery() {  
  return useQuery({  
    queryKey: ["posts", "recent"],  
    queryFn: getRecentPosts,  
  });  
}
```

ATUALIZAÇÃO MANUAL COM REFETCH

O `useQuery` fornece a função `refetch` para solicitar uma nova execução da `queryFn`.

```
const { data: posts, isFetching, refetch } = useRecentPostsQuery();

function handleRefresh() {
  refetch();
}
```

Elemento	Responsabilidade
<code>refetch()</code>	Solicitar uma nova execução
<code>queryFn</code>	Executar a operação assíncrona
<code>isFetching</code>	Indicar que uma execução está em andamento
<code>data</code>	Entregar o resultado mantido no cache



APLICATIVO “PACEUP”

Check-ins de treino com Cloud Firestore

[Acesse o Codelab](#)

O QUE VOCÊ VAI APRENDER?

- configurar o Cloud Firestore no aplicativo;
- representar dados com coleções e documentos;
- consultar check-ins com `getDocs` ;
- publicar check-ins com `addDoc` ;
- sincronizar alterações com `onSnapshot` .



RECOMENDAÇÃO DE LEITURA

NOSQL

Como armazenar os dados de uma aplicação moderna

David Paniz

Comece pelos capítulos:

- **Introdução:** tipos de bancos não relacionais;
- **Bancos orientados a documentos:** relacionamentos, documentos embutidos e desnormalização.

Firestore é uma tecnologia específica.

O modelo orientado a documentos é um conceito mais amplo.

LICENÇA

Este material é disponibilizado sob a licença Creative Commons Atribuição 4.0 Internacional (CC BY 4.0).

  Atribuição 4.0 Internacional

Autor: Felipe Cavalcante Vilella de Oliveira, 2026